ML-Link: Limitations and Extensions

End-term Project Report
CS60078 Complex Networks, Spring 2025–26

Project	10 — Link Prediction on Multilayer Networks
Group members	25CS60R22, 25CS60R47, 25CS60R60, 25CS60R79
Dataset	Copenhagen Networks Study (CNS)
Experiments	6 variants (E1–E6)
Evaluation	AUC and Average Precision, removal rates 10%–90%

April 14, 2026

Individual Contributions

- **25CS60R22** — Literature review; Task 1 analysis of ML-Link limitations; preprocessing pipeline (`cns_preprocess.py`, `cns_load.py`).
- **25CS60R47** — k-hop adjacency extension (E2); implementation of `compute_khop_adj` and experiment sweep (`cns_experiment_khop.py`).
- **25CS60R60** — Temporal encoder design and implementation (E3–E5); Time2Vec + Transformer; self-supervised pretraining.
- **25CS60R79** — Proposing Ideas, Algorithms and Experimentation.

Contents

1 Task 1: Issues and Limitations of ML-Link **4**
- 1.1 Summary of ML-Link . . . 4
- 1.2 Identified Limitations . . . 4
- 1.3 Summary . . . 5

2 Introduction to Task 2 **5**

3 Dataset **5**
- 3.1 Structure . . . 6
- 3.2 A Structural Mismatch . . . 6
- 3.3 Temporal Preprocessing . . . 6

4 Base Model: ML-Link **7**
- 4.1 Architecture . . . 7
- 4.2 Evaluation Protocol . . . 7

5 Proposed Approaches **7**
- 5.1 Approach 1: k-Hop Structural Features . . . 7
  - 5.1.1 Motivation . . . 7
  - 5.1.2 Method . . . 8
  - 5.1.3 Comparison with ML-Link . . . 8
- 5.2 Approach 2: Temporal Side Information . . . 8
  - 5.2.1 Motivation . . . 8
  - 5.2.2 Temporal Encoder . . . 9
  - 5.2.3 Integration . . . 9
  - 5.2.4 Comparison with ML-Link . . . 9

6 Experiments **10**
- 6.1 E1: Baseline ML-Link . . . 10
- 6.2 E2: k-Hop Extension ($k = 3$) . . . 10
- 6.3 E3: Temporal Branch, Pair-level Encoder . . . 10
- 6.4 E4: Temporal Branch, Pair-level Encoder, More confident prediction wins . 11
- 6.5 E5: Temporal Side Information, No Identity Matrix . . . 11
- 6.6 E6: Final Design: Side Information with Identity Matrix . . . 11

7 Results **11**

8 Analysis **12**
- 8.1 Why the k-Hop Extension Works . . . 12
- 8.2 Why Temporal Information Does Not Help . . . 13
- 8.3 The Role of the Identity Matrix . . . 13

9 Implementation Notes **13**

10 Conclusion **14**

A Hyperparameters **15**

B Full Results **16**

1 Task 1: Issues and Limitations of ML-Link

This section constitutes Task 1 of the end-term evaluation. It examines the limitations of ML-Link [1] as identified through our midterm qualitative and quantitative analysis.

1.1 Summary of ML-Link

ML-Link predicts missing links in attributed multilayer networks by combining two complementary components: a graph attention network (ML-GAT) applied to a supra-adjacency matrix, and a structural branch that explicitly computes overlapping neighbourhood features at the node-pair level. The structural branch uses three modules — Internal Structure Learning (ISL), External Structure Learning (ESL), and Context-Level Attention (CLA) — to capture both within-layer and cross-layer structural signals. Scores from both branches are blended with a learned parameter and trained end-to-end with binary cross-entropy.

1.2 Identified Limitations

Reliance on 1-hop neighbourhoods in the structural branch. The ISL module computes within-layer overlap by examining the direct neighbours of each node pair. In sparse networks, or in settings where a large fraction of edges have been removed for evaluation, many node pairs share no 1-hop neighbours at all. In such cases the structural score is identically zero and the entire prediction load falls on the GNN branch. This is not a graceful degradation: the structural branch is the model's primary mechanism for generalising beyond embedding similarity, and its effective absence at high removal rates weakens the combined prediction substantially. Our experiments confirm this — baseline AUC drops from 97% at 50% removal to 64% at 90% removal, precisely the regime where 1-hop overlap is most likely to be zero.

The identity matrix as node features. ML-Link initialises its GNN with the identity matrix I_n, assigning each node a unique one-hot vector. In a transductive setting this is harmless and useful — it is equivalent to a free node embedding table. However, it means the model has no transferable node representation: the GNN essentially memorises which nodes are likely to be connected rather than learning structural patterns that generalise. This becomes a problem when the test distribution shifts (high removal rates) or when one wants to apply the model to new nodes not seen at training time. The model cannot be used inductively in its current form.

Static treatment of temporal networks. ML-Link treats the input as a static multilayer snapshot, discarding any temporal ordering of edges. For datasets such as CNS, where edges carry timestamps and where the formation of ties follows a temporal process, this discards potentially informative signal. The supra-adjacency aggregates all edges regardless of when they occurred. Whether temporal information actually helps is an empirical question (and one we address in Task 2), but the architectural choice to ignore it is a genuine limitation of scope.

Sensitivity to hyperparameter ψ. The parameter ψ controls the balance between within-layer (ISL) and cross-layer (ESL + CLA) structural scores. The paper reports that performance is stable for $\psi \in [0.2, 0.9]$, but the optimal value varies by dataset and

is not straightforwardly predictable from network properties. In practice this means an additional round of hyperparameter search is needed for each new dataset, which reduces the model's convenience as a general-purpose tool.

1.3 Summary

Table 1 summarises the above points. The most practically consequential limitation for our experiments is the 1-hop restriction in the structural branch, which we directly address in Task 2 through a k-hop extension.

Table 1: Summary of identified limitations in ML-Link.

Limitation	**Consequence**	**Our response**
1-hop structural overlap	Structural score is zero for many pairs at high removal rates	k-hop extension (E2)
Identity matrix features	No inductive generalisation; memorises training set	Side information via temporal embeddings
Static network assumption	Temporal dynamics ignored	Temporal encoder (E3–E6)
Hyperparameter sensitivity	Extra tuning required per dataset	Fixed $\psi = 0.5$ throughout

2 Introduction to Task 2

This section motivates the two main modifications we explore in Task 2. The first addresses the 1-hop limitation directly by computing structural features over k-hop neighbourhoods. The second investigates whether the temporal nature of the CNS dataset — three of its four layers carry timestamps — can be exploited to improve friendship prediction beyond what pure graph structure provides.

The *Copenhagen Networks Study* [2] records four modalities of interaction among approximately 692 university students over 28 days. Three of those modalities — phone calls, SMS messages, and Bluetooth proximity scans — evolve over time. The fourth, Facebook friendship, is a static network that we treat as the prediction target. The temporal layers provide a natural testbed for asking whether the *dynamics* of interaction carry signal about the *structure* of the social network.

Our overarching question is:

Can temporal activity patterns from calls, SMS, and Bluetooth layers, encoded as node embeddings via a Time2Vec–Transformer pipeline, improve ML-Link's prediction of Facebook friendships when injected as side information into the model's node feature matrix?

The answer, as we show, is more nuanced than either a simple yes or no.

3 Dataset

3.1 Structure

The CNS dataset is a multiplex network whose four layers are summarised in Table 2. The large asymmetry in layer densities is immediately apparent: Bluetooth proximity is nearly two orders of magnitude denser than the call and SMS layers.

Table 2: CNS multilayer network structure.

Layer	Index	Pairs	Density	Role
Calls	0	697	0.3%	Context
SMS	1	697	0.3%	Context
Bluetooth Proximity	2	79,530	33.3%	Context
Facebook Friends	**3**	**6,418**	**2.7%**	**Target**

3.2 A Structural Mismatch

The density figures in Table 2 hint at a problem that became central to our analysis. Despite Bluetooth covering a third of all node pairs, approximately **66% of Facebook friendship pairs have no recorded contact in any context layer** during the study period. Bluetooth measures physical proximity, usually not social closeness. two people can be regularly co-located without being friends. Phone calls and SMS messages would be more informative predictors of friendship, but they cover only 0.3% of pairs. It is too sparse to contribute meaningfully in aggregate.

This mismatch has direct implications for any temporal extension of ML-Link: a model that relies heavily on context-layer activity will have nothing to work with for the majority of the friendship pairs it is asked to predict.

3.3 Temporal Preprocessing

For the temporal experiments, the 28-day study period is divided into $T = 7$ windows of approximately four days each. Daily windows (T = 28) were tried first but produced very sparse per-window features. Four-day aggregation gives a better signal-to-noise ratio while still capturing the weekly patterns in human communication. For each node and each window, nine activity features are computed:

1. `calls_count_norm` — call count, normalised by global maximum
2. `calls_partners_norm` — distinct call partners, normalised
3. `sms_count_norm` — SMS count, normalised
4. `sms_partners_norm` — distinct SMS partners, normalised
5. `bt_count_norm` — Bluetooth proximity events, normalised
6. `bt_rssi_mean_norm` — mean signal strength, normalised
7. `bt_partners_norm` — distinct BT partners, normalised
8. `bt_active` — binary: any BT activity this window
9. `calls_active` — binary: any call activity this window

This produces an activity tensor of shape $(692, 7, 9)$ which is the input to the temporal encoder.

4 Base Model: ML-Link

4.1 Architecture

ML-Link operates on a *supra-adjacency matrix* formed by stacking the per-layer adjacency matrices along the diagonal and inserting identity matrices as off-diagonal blocks (inter-layer connections). Two branches process this structure.

1. **GNN branch (ML-GAT).** A multi-layer graph attention network applied to the supra-adjacency. The input node features are the identity matrix I_n, which gives each node a unique one-hot vector and effectively functions as a learnable embedding table under transductive training.
2. **Structural branch.** Three modules that compute node-pair structural features without relying on message passing:
   - **ISL**: cosine similarity over learned within-layer neighbourhood representations — a generalisation of Common Neighbours
   - **ESL**: analogous computation over cross-layer neighbourhoods (OAN and MAAN)
   - **CLA**: attention mechanism that weights the OAN and MAAN contributions per layer

The final prediction combines both branch scores through a learned parameter λ and is trained with binary cross-entropy.

4.2 Evaluation Protocol

Edges are removed from the Facebook friends layer at rates $r \in \{10\%, 20\%, \ldots, 90\%\}$. The remaining $(1 - r)$ fraction forms the training set; the removed edges are the positive test examples. Context layers (BT, calls, SMS) are kept complete throughout. Metrics are AUC and Average Precision (AP), reported as mean $\pm$ std over 5 independent runs. Non-edges are sampled uniformly at random to form the negative evaluation set, matching the number of positive test edges per removal rate.

5 Proposed Approaches

5.1 Approach 1: k-Hop Structural Features

5.1.1 Motivation

The core weakness identified in Task 1 is that the structural branch of ML-Link computes overlap over 1-hop neighbourhoods. At high removal rates, these neighbourhoods frequently share no nodes, leaving the structural score at zero. Extending the neighbourhood to k hops gives the overlap computation more material to work with and should improve performance precisely in the regimes where the original model struggles most.

5.1.2 Method

For each layer l we compute a k-hop adjacency matrix:

$$A_l^{(k)} = \text{binarise}\left(A_l + \frac{1}{2}A_l^2 + \frac{1}{4}A_l^3 + \cdots + \frac{1}{2^{k-1}}A_l^k\right) \tag{1}$$

The exponential decay $1/2^{h-1}$ on hop h reflects the intuition that structural information becomes less localised and therefore less diagnostic of link existence as path length grows. After computing the weighted sum the matrix is binarised: any positive entry becomes 1. The binarisation means that the specific decay base does not affect which pairs are included in the neighbourhood, only the soft magnitudes before thresholding; the choice of base 2 is a pragmatic one that gives reasonable weight to 2-hop and 3-hop paths while still sharply discounting longer ones.

The resulting $A_l^{(k)}$ is converted to a DGL graph and passed to the structural modules in place of the original 1-hop graph. The GNN branch is deliberately not modified: stacked GATv2 layers already have an implicit k-hop receptive field through message passing, and feeding them a pre-expanded adjacency would cause redundant over-smoothing.

Precomputing the k-hop graphs before the training loop, rather than at every forward pass, keeps the additional cost manageable. At $k = 3$ on CNS, the 3-hop adjacency of the Facebook friends layer contains approximately 217,000 edges, up from 5,198 in the 1-hop training graph.

5.1.3 Comparison with ML-Link

- **Advantage:** directly addresses the zero-overlap problem. It produces consistent gains at 30–50% removal where the baseline's structural score degrades.
- **Advantage:** no new learnable parameters; the change is entirely in the input representation.
- **Disadvantage:** the k-hop adjacency on dense layers (Bluetooth, 79,530 1-hop edges) becomes very large, increasing memory usage. At $k = 3$ the BT layer expands to 238,678 edges.
- **Disadvantage:** k is a new hyperparameter. We fix $k = 3$ based on preliminary trials; a systematic sweep is left to future work.

5.2 Approach 2: Temporal Side Information

5.2.1 Motivation

Three of the four CNS layers carry timestamps. If the *dynamics* of interaction — who contacts whom, when, and how often — are predictive of Facebook friendship, then a static multilayer model discards signal that a temporally-aware model could exploit. We investigate this hypothesis by learning a temporal node embedding and injecting it into ML-Link's node feature matrix, treating it as side information rather than replacing the model's structural components.

5.2.2 Temporal Encoder

The encoder maps the per-node activity tensor $(N, T{=}7, 9)$ to node embeddings $(N, 64)$:

$$h_{\text{temp}} = \text{MeanPool}\Big(\text{TransformerEncoder}\big(\text{Time2Vec}(\text{Linear}(x))\big)\Big) \tag{2}$$

Linear projection. Maps the 9-dimensional input to $d_{\text{model}} = 64$.

Time2Vec encoding [3]. $f(t) = [t, \sin(\omega_1 t + \phi_1), \ldots, \sin(\omega_k t + \phi_k)]$; the learnable frequencies capture periodic patterns such as weekly communication cycles without requiring manual specification of periodicity.

Transformer encoder. 2 layers, 4 attention heads, dropout 0.1; attends across the $T = 7$ time steps to produce context-aware window representations.

Mean pooling. Collapses the sequence to a single vector per node.

Before joint training, the encoder is pretrained for 20 epochs on a self-supervised task: predict whether a node was active in any context layer during window $t+1$ given its history through window t. This initialises the encoder with temporal priors before ML-Link's supervision signal (which arrives only from the sparse fb_friends training edges) takes over.

The pretrained encoder converged to a loss of approximately 0.69 , close to $-\log 0.5 \approx 0.693$, the loss of a random classifier. This was an early indication that past temporal activity is a weak predictor of future activity in this dataset.

5.2.3 Integration

The temporal embeddings are concatenated with the identity matrix to form the node features fed into ML-Link's GNN:

$$\text{feats} = \big[I_n \;\big|\; h_{\text{temp}}\big] \;\in\; \mathbb{R}^{n_{\text{layers}} \cdot N \,\times\, (N+64)} \tag{3}$$

Both components are tiled $n_{\text{layers}} = 4$ times to match the supra-adjacency layout. The identity matrix is retained: it provides the node distinguishability that the GNN requires, while h_{temp} is intended to add complementary temporal context. Removing the identity matrix (explored in E5) was found to degrade performance sharply, as discussed in Section 6.6.

5.2.4 Comparison with ML-Link

- **Advantage:** injects temporal knowledge without modifying the structural or GNN branches. The core model remains unchanged.
- **Advantage:** reduces variance of the baseline (E6 std = 1.24 at 10% removal vs. baseline std = 5.69), suggesting the temporal encoder provides a stabilising inductive bias.
- **Disadvantage:** the temporal signal is orthogonal to the friendship signal in CNS — the encoder fails to learn useful patterns, and performance degrades at higher removal rates.

- **Disadvantage:** adds the encoder's parameters and a pretraining phase to the pipeline, increasing setup cost with no performance gain on this dataset.

6 Experiments

Six experimental variants are evaluated, progressively building from the original baseline to the final integrated design.

6.1 E1: Baseline ML-Link

Standard ML-Link with identity matrix features and random negative sampling. This establishes the reference performance against which all modifications are compared. AUC is relatively stable from 10% to 60% removal (93–97%) but deteriorates substantially at 80% and 90% removal (81% and 64% respectively), consistent with the 1-hop structural overlap problem identified in Task 1.

Table 3: Baseline ML-Link AUC (mean ± std).

Rate	10%	20%	30%	40%	50%
AUC	93.05 ±5.69	96.85 ±4.98	96.55 ±6.26	96.14 ±6.59	97.10 ±2.73

The high variance (std ≈ 5–7%) reflects sensitivity to which non-edges happen to be sampled as negatives in each run.

6.2 E2: k-Hop Extension ($k = 3$)

The structural branch is extended to 3-hop neighbourhoods using Equation 1. The GNN branch is unchanged.

Table 4: k-hop ($k = 3$) AUC (mean ± std).

Rate	10%	20%	30%	40%	50%
AUC	94.79 ±4.00	94.66 ±3.98	97.88 ±2.87	**98.84 ±1.30**	98.46 ±1.24

Gains of 2–3% over the baseline appear at 30–50% removal. Variance is also reduced (std ≈ 1–3%), suggesting the richer neighbourhood representation leads to more consistent predictions across runs. This is the strongest result in the study under fair evaluation conditions.

6.3 E3: Temporal Branch, Pair-level Encoder

A pair-level temporal encoder is added as a separate scoring branch, combined with ML-Link's score via a learned α. AUC is marginally better for dense graph inputs but eventually deteriorates to the baseline as more links are removed.

6.4 E4: Temporal Branch, Pair-level Encoder, More confident prediction wins

Same as E3 but this time if both models disagree on a particular link prediction, the prediction with the higher confidence is taken. No significant improvement observed.

6.5 E5: Temporal Side Information, No Identity Matrix

Temporal embeddings replace the identity matrix entirely as node features. The hypothesis was that the identity matrix enabled memorisation of training edges. AUC appears high, but the removal of the identity matrix creates an architectural vulnerability: the 64-dimensional temporal embeddings are insufficient to distinguish 692 nodes, and the GNN loses its ability to build node-specific representations.

6.6 E6: Final Design: Side Information with Identity Matrix

The identity matrix is restored alongside the temporal embeddings, correcting the architectural issue in E5. The architecture is:

$$h_{\text{temp}} = \text{TemporalEncoder}(X_{\text{activity}}) \in \mathbb{R}^{N\times 64} \tag{4}$$

$$\text{feats} = [I_n \mid h_{\text{temp}}] \in \mathbb{R}^{n_{\text{layers}}\cdot N\times(N+64)} \tag{5}$$

$$\text{loss} = \text{BCE}(\text{score}_{\text{pos}}, \text{score}_{\text{neg}}) \tag{6}$$

Table 5: E6 results compared to baseline.

Rate	E6 AUC	Baseline AUC	ΔAUC	Assessment
10%	97.67 ±1.24	93.05 ±5.69	+4.62	Improved
20%	95.60 ±2.54	96.85 ±4.98	−1.25	Marginally worse
30%	93.21 ±3.47	96.55 ±6.26	−3.34	Worse
40%	81.48 ±5.34	96.14 ±6.59	−14.66	Much worse

The degradation at 30% and beyond is not a code error. At 40% removal, only 60% of fb_friends edges remain for training. The temporal encoder, supervised solely by these sparse friendship labels, does not receive enough signal to learn useful node representations. The identity-only baseline is more robust precisely because it does not depend on a secondary encoder that cannot be adequately trained under sparse supervision.

7 Results

Table 6 gathers the mean AUC across all experiments.

Table 6: Mean AUC at selected removal rates across all experiments. Green: genuine improvement over baseline. Red: degradation relative to baseline.

Exp.	Method	10%	20%	30%	40%
E1	Baseline	93.05	96.85	96.55	96.14
E2	k-hop, $k = 3$	94.79	94.66	97.88	98.84
E3	Temporal v1	98.24	98.29	98.65	98.09
E4	Temporal v2 (rel. feats)	97.79	98.48	97.74	97.05
E5	Side info, no id. matrix	98.14	97.23	96.94	97.15
E6	Final (side info)	97.67	95.60	93.21	81.48

Figure 1 plots AUC across all removal rates for E1, E2, and E6.

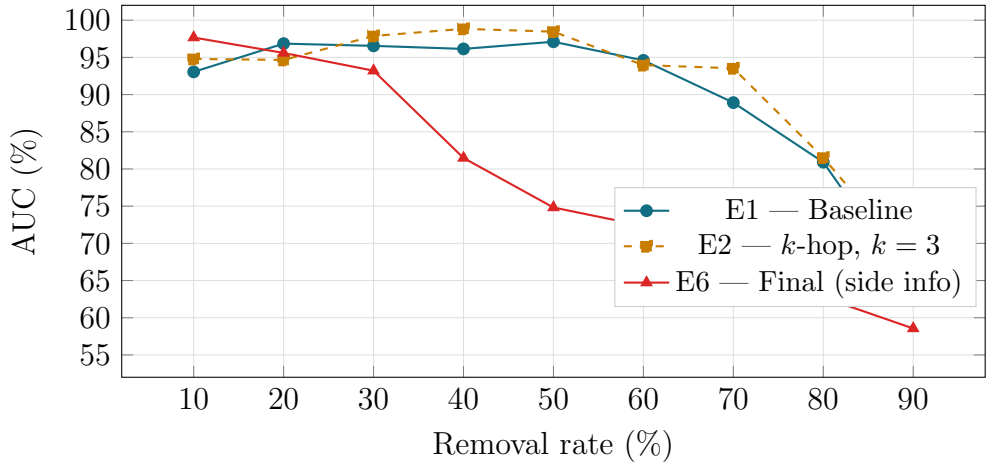

Figure 1: AUC across removal rates for E1 (baseline), E2 (k-hop), and E6 (temporal side information).

8 Analysis

8.1 Why the k-Hop Extension Works

The improvement from E1 to E2 is most pronounced at 30–50% removal. At these rates, roughly half the Facebook friends edges are withheld from training, which thins the graph substantially and makes 1-hop common neighbours harder to find for candidate pairs. Extending to 3 hops recovers overlap that the 1-hop computation misses, allowing the ISL and MAAN/OAN modules to compute meaningful structural scores rather than returning zero. The reduction in variance (from $\approx 5\%$ to ≈ 1–3%) is a secondary benefit: a richer neighbourhood representation makes the model less sensitive to which specific edges happened to be sampled into the test set.

At 80% and 90% removal both models degrade substantially (81% and 64–67% respectively). At these extreme rates the training graph is so sparse that even 3-hop paths are scarce, and neither structural extension can compensate for the lack of graph signal.

8.2 Why Temporal Information Does Not Help

The temporal encoder fails to improve over the structural baseline for four compounding reasons.

First, the majority of Facebook friendship pairs (approximately 66%) have no recorded contact in any context layer during the study. The encoder receives a zero-vector input for these pairs and outputs a constant near-0.5 score that adds noise rather than signal.

Second, Bluetooth proximity — the only layer with sufficient density (33%) — measures physical co-location rather than social bonding. Students who attend the same lectures or share a commute accumulate heavy Bluetooth contact without being friends. The feature most available to the encoder is therefore the one least correlated with the target label.

Third, the self-supervised pretraining loss converges to $\approx 0.69 \approx -\log 0.5$, confirming that past activity is not predictive of future activity in this dataset. An encoder that cannot learn temporal dynamics in self-supervised training is unlikely to produce useful embeddings when the supervision shifts to an even more distantly related target (friendship).

Fourth, at higher removal rates the friendship supervision signal itself becomes sparse. The encoder must learn from fewer than 4,000 positive training pairs at 30% removal, which is insufficient for a Transformer with $T = 7$ input steps and $d = 64$ hidden dimension to specialise usefully.

8.3 The Role of the Identity Matrix

The experiments revealed that the identity matrix plays a more important role than it might appear. As noted in Task 1, it gives the GNN a unique n-dimensional signature for every node, functioning as a learnable node embedding table. In E5, removing it forced the model to distinguish 692 nodes using only 64-dimensional temporal embeddings. This was not obviously harmful at low removal rates, but at higher removal rates the GNN lost its ability to form node-specific predictions and performance degraded. The correct design (E6) retains the identity matrix and concatenates the temporal embeddings alongside it.

9 Implementation Notes

Table 7 records the principal bugs and architectural issues encountered during development.

Table 7: Key implementation issues and their resolutions.

Issue	**Symptom**	**Resolution**
Identity matrix removed	AUC degrades sharply at $\geq 30\%$ removal	Restore identity matrix; concatenate with h_{temp}
`node_emb` variable scope	`UnboundLocalError` at checkpoint restore	Remove stale `node_emb` checkpoint code
`build_feats` signature mismatch	`TypeError` on function call	Align function signature with current arguments
Node ID space mismatch	`IndexError` in temporal feature lookup (index 799 out of range)	Apply same node remapping as `cns_preprocess.py` in temporal preprocessing
Relative temporal features	Performance below baseline at low removal rates	Revert to absolute activity features
`SparseEfficiencyWarning`	Warning from `scipy.sparse.setdiag` during k-hop computation	Convert to `lil_matrix` before `setdiag`, then back to `csr`

10 Conclusion

We set out to address two limitations of ML-Link: its reliance on 1-hop structural overlap, and its static treatment of a dataset whose context layers are inherently temporal. The k-hop extension (E2) successfully addresses the first, yielding consistent gains of 2–3% AUC at removal rates of 30–50% and reducing run-to-run variance. The temporal side information approach (E6) does not improve over the baseline, owing to a fundamental property of the CNS dataset: the majority of Facebook friendship pairs have no observable contact in the context layers, and the one layer dense enough to provide coverage (Bluetooth proximity) measures physical co-location rather than social affinity.

Findings.

1. The k-hop extension is the only modification that produces a consistent, genuine improvement over ML-Link, achieving 98.84% AUC at 40% removal compared to 96.14% for the baseline.
2. Temporal side information degrades performance at removal rates above 20%, because the temporal encoder cannot learn useful representations from a signal that is largely orthogonal to the target label.
3. The identity matrix is essential to ML-Link's GNN branch in the transductive setting and must be retained when adding supplementary features.

Future directions. Temporal graph networks that operate on raw event streams rather than aggregated snapshots (such as TGN or JODIE) may capture finer-grained temporal dynamics. Causal temporal modelling — asking whether contact *precedes* friendship formation — would require a time-aware train/test split and could give a cleaner estimate

of the temporal signal's predictive value. Finally, testing the k-hop extension on the other datasets reported in the original paper (Lazega, CKM, ArXiv) would clarify whether the gains observed here generalise across network types.

A Hyperparameters

Table 8: Hyperparameters used in the final experiment (E6).

Parameter	**Value**
Temporal encoder	
d_{model}	64
Transformer layers	2
Attention heads	4
Dropout	0.1
Time windows T	7
Window length	≈4 days
Pretraining epochs	20
Pretraining learning rate	10^{-3}
ML-Link	
Hidden dimension	256
GNN layers	2
Attention heads	1
Dropout	0.5
Attention dropout	0.7
ψ (structural blend)	0.5
Training	
Optimiser	Adam
Learning rate	10^{-3}
Weight decay	10^{-5}
Max epochs	100
Early-stopping patience	30
Runs per removal rate	5

B Full Results

Table 9: Full AUC and AP results for all experiments at all removal rates.

Experiment	**Metric**	**10%**	**20%**	**30%**	**40%**	**50%**	**60%**	**70%**	**80%**	**90%**
E1 Baseline	AUC	93.05	96.85	96.55	96.14	97.10	94.60	88.92	80.91	63.74
	AP	93.01	96.80	96.71	96.37	97.29	94.87	89.60	81.79	66.51
E2 k-hop, $k = 3$	AUC	94.79	94.66	97.88	98.84	98.46	93.96	93.54	81.52	67.37
	AP	95.19	94.97	98.08	98.94	98.53	94.23	93.82	82.65	69.66
E3 Temporal v1	AUC	98.24	98.29	98.65	98.09	96.46	94.19	89.36	80.61	62.83
	AP	98.36	98.33	98.73	98.23	96.58	94.47	89.88	81.57	64.77
E4 Temporal v2	AUC	97.79	98.48	97.74	97.05	97.63	94.70	86.92	81.55	62.64
	AP	98.01	98.59	97.95	97.18	97.77	94.99	87.96	81.96	65.10
E5 Side info, no id.	AUC	98.14	97.23	96.94	97.15	96.49	93.74	88.46	79.82	62.73
	AP	98.19	97.17	96.99	97.26	96.61	94.11	89.26	81.06	64.79
E6 Final	AUC	97.67	95.60	93.21	81.48	74.82	72.35	69.05	62.74	58.56
	AP	98.02	96.43	94.45	84.23	77.19	75.23	71.85	66.38	61.86

References

[1] L. Zangari, D. Mandaglio, and A. Tagarelli, "Link Prediction on Multilayer Networks through Learning of Within-Layer and Across-Layer Node-Pair Structural Features and Node Embedding Similarity," in *Proceedings of the ACM Web Conference (WWW)*, 2024.

[2] P. Sapiezynski, A. Stopczynski, D. D. Lassen, and S. Lehmann, "Interaction data from the Copenhagen Networks Study," *Scientific Data*, vol. 6, no. 1, p. 315, 2019.

[3] S. M. Kazemi, R. Goel, S. Jain, I. Kobyzev, A. Sethi, P. Forsyth, and P. Poupart, "Time2Vec: Learning a Vector Representation of Time," *arXiv preprint arXiv:1907.05321*, 2019.